

Deep Learning for High-Dimensional PDEs via Forward-Backward Stochastic Differential Equations A NAIS-Net Approach for Derivatives Pricing

Ionut Nodis*

CQF Institute

Fitch Learning, London, UK

January 2026

Abstract. The pricing of financial derivatives in high dimensions reduces, in essence, to the problem of solving second-order parabolic partial differential equations. For dimensions beyond a handful, classical numerical methods succumb to the curse of dimensionality; the computational cost grows exponentially, rendering grid-based approaches infeasible. In this paper, we develop and analyse a deep learning methodology for solving such equations in the context of derivative pricing. Our solver, built upon the Non-Autonomous Input-Output Stable Network (NAIS-Net) architecture with sinusoidal activations, achieves sub-1% relative errors on benchmark problems in dimension $d = 100$, a regime entirely inaccessible to traditional methods. We present numerical experiments on the Black-Scholes-Barenblatt, Hamilton-Jacobi-Bellman, and XVA pricing equations, and discuss practical considerations for deployment in production risk management systems.

Keywords: deep learning, backward stochastic differential equations, high-dimensional PDEs, derivatives pricing, neural networks, curse of dimensionality

1 Introduction

Partial differential equations occupy a peculiar position in applied mathematics: they are at once the natural language for describing continuous dynamical systems and, in high dimensions, among the most computationally intractable objects one can encounter. In quantitative finance, such equations arise with uncomfortable frequency — in derivative pricing, optimal execution, and the risk management of large portfolios. The difficulty is not conceptual but computational, and it is severe. The root of the problem is the *curse of dimensionality*. Classical numerical methods — finite differences, finite elements, spectral methods — discretise the spatial domain on a grid. The computational cost scales as $\mathcal{O}(N^d)$, where N is the number of grid points per dimension and d is the spatial dimension.

*Correspondence: ionutnodis@outlook.com

For modest d , this is manageable. For $d \gtrsim 4$, it becomes prohibitive. To appreciate the scale of the difficulty, consider pricing a basket option on $d = 100$ underlying assets. A discretisation of $N = 100$ points per dimension yields a grid of 10^{200} nodes, a number that exceeds the estimated count of atoms in the observable universe by some one hundred and twenty orders of magnitude. The problem is not merely expensive; it is absurd.

This curse is by no means confined to problems in finance. The Schrödinger equation in quantum many-body physics scales with three times the number of particles. Hamilton-Jacobi-Bellman equations in dynamic programming grow with the number of agents or decision variables. Portfolio optimisation, resource allocation, and countless other problems exhibit the same dimensional explosion. For decades, these high-dimensional PDEs have stood as a barrier, not to understanding, but to computation.

The path forward, it turns out, was laid long ago. The connection between partial differential equations and stochastic processes has a history stretching back to the mid-twentieth century. In 1949, Kac established a remarkable correspondence between parabolic PDEs and expectations over Brownian paths, building upon Feynman’s path integral formulation of quantum mechanics from the preceding year. The resulting Feynman-Kac formula asserts that *the solution to certain linear parabolic PDEs can be expressed as an expected value over stochastic trajectories*. This insight opened the door to Monte Carlo methods for PDE solving — methods that circumvent spatial discretisation entirely by sampling paths rather than enumerating grid points.

A decisive theoretical advance came in 1990, when Pardoux and Peng proved the existence and uniqueness of solutions to nonlinear backward stochastic differential equations [2]. Their work extended the classical Feynman-Kac correspondence to a far broader class of semilinear parabolic PDEs, establishing what is now termed the *nonlinear Feynman-Kac formula*. In this framework, the PDE solution $u(t, x)$ and its gradient $\nabla u(t, x)$ are represented not as static functions on a grid, but as adapted stochastic processes (Y_t, Z_t) satisfying a coupled forward-backward system. The subsequent development of BSDE theory [3, 4] revealed deep structural connections between stochastic control, mathematical finance, and nonlinear PDEs, connections that would prove essential to the methods developed in this paper.

Recent advances in deep learning have furnished the computational machinery to exploit these connections. The essential idea, introduced by Han, Jentzen, and E [5, 6], is to reformulate the high-dimensional PDE as a stochastic control problem and parameterise the unknown solution and its gradient using neural networks. The universal approximation capabilities of deep networks, combined with the probabilistic structure of the BSDE formulation, yield a method that scales gracefully with dimension, or at least, far more gracefully than any grid-based alternative.

There is, however, a caveat. For such methods to be viable in practice, particularly in risk-sensitive domains where fortunes may depend on the accuracy of a computed hedge ratio, the underlying numerical scheme must be stable and robust. Small perturbations in inputs or training dynamics must not cascade into large deviations in the solution.

1.1 Contributions

The present work makes the following contributions:

1. **Robust Implementation:** A production-quality Python library implementing Deep FBSDE solvers with multiple network architectures (feedforward, NAIS-Net) and activation functions.
2. **Convergence Analysis:** Rigorous theoretical analysis establishing convergence guarantees under verifiable assumptions, with explicit error bounds.
3. **Extensive Benchmarking:** Numerical experiments on Black-Scholes-Barenblatt, multi-asset basket options, and Hamilton-Jacobi-Bellman equations across dimensions $d \in \{10, 50, 100\}$.
4. **Practical Guidance:** Discussion of hyperparameter selection, training stability, and deployment considerations for practitioners.

1.2 Paper Organisation

The remainder of this paper is organised as follows. Section 2 establishes the mathematical framework connecting PDEs, FBSDEs, and neural network approximation. Section 3 details our implementation methodology. Section 4 presents numerical results across three benchmark problems. Section 6 concludes with reflections on the method’s capabilities and directions for future inquiry.

2 Mathematical Framework

Before proceeding to algorithms and experiments, we must establish the theoretical apparatus that makes high-dimensional PDE solving possible. The story has three parts: first, why classical methods fail; second, how stochastic calculus provides an escape route; and third, why particular neural architectures are required to traverse that route without numerical catastrophe.

2.1 The General Parabolic PDE

We consider a time-dependent state variable $x \in \mathbb{R}^d$, representing the prices of d underlying assets. Our objective is to approximate the function $u(t, x) : [0, T] \times \mathbb{R}^d \rightarrow \mathbb{R}$, which represents the fair value of a derivative contract. The general class of PDEs we seek to solve takes the form:

$$\frac{\partial u}{\partial t} + \frac{1}{2} \text{Tr} \left(\sigma \sigma^\top(t, x) \text{Hess}_x u \right) + \nabla u \cdot \mu(t, x) + f(t, x, u, \sigma^\top \nabla u) = 0, \quad (1)$$

defined on $[0, T] \times \mathbb{R}^d$, subject to the terminal condition $u(T, x) = g(x)$.

Let us parse this equation term by term:

- The term $\frac{\partial u}{\partial t}$ captures the time decay of the derivative’s value — the familiar theta of options pricing.

- The diffusion term $\frac{1}{2}\text{Tr}(\sigma\sigma^\top \text{Hess}_x u)$ encodes the effect of volatility on the price through the curvature of the value function. This is the gamma term responsible for the convexity.
- The drift term $\nabla u \cdot \mu$ represents directional exposure to the underlying assets — the delta.
- The nonlinear driver $f(t, x, u, \sigma^\top \nabla u)$ is where matters become interesting. This term accommodates a vast range of phenomena: funding costs, credit adjustments, transaction costs, and optimal control. It is the source of the nonlinearity that takes us beyond Black-Scholes.

The terminal condition $g(x)$ specifies the derivative's payoff at maturity. For a European call option, $g(x) = (x - K)^+$. For the benchmark problems considered in this paper, g takes various forms designed to test different aspects of the numerical method to assess stability.

2.2 The Nonlinear Feynman-Kac Formula

The classical Feynman-Kac theorem, in its original form, applies to linear parabolic PDEs and expresses the solution as an expected value over Brownian paths. The extension to nonlinear equations requires the machinery of backward stochastic differential equations, developed by Pardoux and Peng in their foundational 1990 paper [2].

The key insight is this: the PDE solution $u(t, x)$ can be represented as the initial value of a stochastic process (Y_t, Z_t) that evolves *backward* in time, coupled to a forward diffusion process X_t that models the asset dynamics. The system takes the form:

$$\begin{cases} dX_t = \mu(t, X_t) dt + \sigma(t, X_t) dW_t, & X_0 = x_0 \\ dY_t = -f(t, X_t, Y_t, Z_t) dt + Z_t^\top dW_t, & Y_T = g(X_T) \end{cases} \quad (2)$$

$$(3)$$

The forward equation (2) is entirely standard — it describes the risk-neutral evolution of asset prices under geometric Brownian motion. The backward equation (3) is where the magic happens. The process Y_t represents the derivative's value at time t , while Z_t represents the hedging portfolio — more precisely, $Z_t = \sigma(t, X_t)^\top \nabla u(t, X_t)$. The terminal condition $Y_T = g(X_T)$ enforces the payoff at maturity. The equation then propagates this information *backward* through time, with the driver f encoding how intermediate values depend on the solution itself. This recursive structure is what allows the framework to handle nonlinear pricing adjustments.

Theorem 2.1 (Nonlinear Feynman-Kac). *Under suitable regularity conditions on μ, σ, f , and g , the solution to the PDE (1) satisfies:*

$$u(t, x) = Y_t \quad \text{and} \quad \sigma(t, x)^\top \nabla u(t, x) = Z_t, \quad (4)$$

where (X_t, Y_t, Z_t) solves the coupled FBSDE system (2)–(3) with $X_t = x$.

This theorem is the theoretical cornerstone of our numerical method. It transforms the problem of solving a PDE on a high-dimensional grid into the problem of simulating a system of stochastic differential equations along sampled paths. The curse of dimensionality is not abolished, the solution still lives in $\mathbb{R}^d$, but it is circumvented. We need only evaluate the solution along the paths that matter, not at every point in space.

2.3 The Manifold Hypothesis and Dimensional Reduction

Why should a neural network with a finite number of parameters be able to approximate a function defined on $\mathbb{R}^{100}$? The answer lies in the *manifold hypothesis*: high-dimensional data typically concentrates on or near a low-dimensional submanifold of the ambient space.

In our setting, this principle takes a precise form. The forward SDE (2) defines a probability measure $\mathbb{P}$ on the space of paths. At any time t , the random variable X_t is distributed according to a measure that, while supported on $\mathbb{R}^d$, concentrates its mass in a region whose effective dimension is far smaller than d . The neural network learns to approximate $u(t, x)$ not everywhere, but on the support of this measure — which is precisely where the pricing problem requires an answer. The loss function makes this explicit:

$$\mathcal{L}(\theta) = \mathbb{E}_{X \sim \mathbb{P}} [|Y_T - g(X_T)|^2] \approx \int_{\mathcal{M}} |u(T, x) - \hat{u}_\theta(T, x)|^2 d\mathbb{P}(x) \quad (5)$$

The solver effectively ignores the vast, empty regions of the ambient space. To use an analogy: solving the PDE with a grid is like mapping the temperature of the entire atmosphere by placing thermometers every ten metres at all altitudes, mostly measuring empty sky. On a submanifold, the approach is akin to attaching thermometers to a flock of migrating birds. We learn the temperature map along the flight paths, obtaining an accurate model of the relevant atmosphere without wasting resources on the void.

2.4 Dynamical Systems and NAIS-Net Stability

Standard Deep FBSDE implementations typically use residual networks to approximate the gradient Z_t . However, there is a subtlety here that deserves attention. A residual network with update rule $x_{k+1} = x_k + f(x_k)$ can be viewed as the forward Euler discretisation of the autonomous dynamical system $\dot{x} = f(x(t))$. In deep networks, this discretisation can become unstable, leading to the familiar pathologies of exploding or vanishing gradients.

For financial applications, where the accuracy of computed hedge ratios has direct monetary consequences, such instabilities are unacceptable. We therefore employ the Non-Autonomous Input-Output Stable Network (NAIS-Net) architecture [7, 8], which imposes algebraic constraints on the network weights to guarantee stability. The NAIS-Net residual block takes the form:

$$h_{k+1} = h_k + \Delta t \cdot \sigma(Ah_k + Bu + b) \quad (6)$$

where u is the input (the state X_t). Stability is enforced by requiring the matrix A to be symmetric negative definite, parameterised as $A = -R^\top R - \epsilon I$. This constraint

ensures that the mapping from asset prices to hedge ratios is contractive and bounded. The network cannot amplify perturbations; it can only attenuate them. This dynamical systems perspective, viewing the network as a discretised differential equation with guaranteed stability properties, is what allows the solver to converge reliably even for stiff, high-dimensional problems.

3 Methodology

Having established the theoretical foundations, we now turn to the practical question of implementation. The numerical resolution of the high-dimensional nonlinear PDE (1) proceeds by approximating the solution to the equivalent FBSDE system. We construct a mesh-free approximation scheme $\mathcal{A}^\theta : \mathbb{R}^d \times [0, T] \rightarrow \mathbb{R}$ parameterised by neural network weights $\theta \in \mathbb{R}^p$. The methodology unfolds in four stages: temporal discretisation of the stochastic processes, justification of the solution space via the manifold hypothesis, parameterisation of the control processes via NAIS-Net, and formulation of the learning problem as minimisation over the Wiener space.

3.1 Temporal Discretisation and The Euler-Maruyama Map

Let $\pi = \{0 = t_0 < t_1 < \dots < t_N = T\}$ be a uniform partition of the time interval $[0, T]$ with mesh size $\Delta t = T/N$. We define the discrete-time filtration $\mathbb{F}^\pi = \{\mathcal{F}_{t_n}\}_{n=0}^N$ generated by the increments of the Brownian motion $\Delta W_n = W_{t_{n+1}} - W_{t_n} \sim \mathcal{N}(0, \Delta t I_d)$.

The forward process X_t is approximated by the Euler-Maruyama scheme, defined recursively for $n = 0, \dots, N-1$:

$$X_{n+1}^\pi = X_n^\pi + \mu(t_n, X_n^\pi)\Delta t + \sigma(t_n, X_n^\pi)\Delta W_n, \quad X_0^\pi = \xi, \quad (7)$$

where $\xi \sim \mathcal{U}(\Omega)$ is a distributed initial condition required for global convergence.

The backward process (Y_t, Z_t) is discretised by approximating the integral form of the BSDE. Let $Y_n^\pi \approx u(t_n, X_n^\pi)$ and $Z_n^\pi \approx \sigma(t_n, X_n^\pi)^\top \nabla u(t_n, X_n^\pi)$. In the Deep BSDE framework [5], we treat $Y_0 = \theta_{Y_0}$ as a trainable parameter and evolve the value process forward:

$$Y_{n+1}^\pi = Y_n^\pi - f(t_n, X_n^\pi, Y_n^\pi, Z(t_n, X_n^\pi; \theta))\Delta t + Z(t_n, X_n^\pi; \theta)^\top \Delta W_n. \quad (8)$$

Here, the gradient process Z_n^π is not an independent variable but is strictly coupled to the value function via the Malliavin derivative relationship $Z(t, x; \theta) = \sigma(t, x)^\top \nabla_x \mathcal{U}(t, x; \theta)$.

3.2 The NAISNet Architecture

To approximate the high-dimensional gradient fields required by our solver, we employ the Non-Autonomous Input-Output Stable Network (NAISNet). This architecture represents a fundamental departure from standard feedforward networks or vanilla residual networks. While standard ResNets can be interpreted as discretisations of *autonomous* ordinary differential equations of the form $\dot{h}(t) = f(h(t))$, the NAISNet models network depth as the time evolution of a *non-autonomous* dynamical system driven continuously by the market state X_0 .

3.2.1 Mathematical Formulation: The Non-Autonomous ODE

Standard deep networks often suffer from signal attenuation — a kind of “catastrophic forgetting” of the boundary conditions as depth increases.

From a spectral perspective, standard ResNets are susceptible to *Jacobian eigenvalue collapse*. As depth $L \rightarrow \infty$, the singular values of the product Jacobian $\prod_{k=1}^L (I + \frac{\partial f_k}{\partial h_k})$ tend to concentrate around zero or infinity, destroying the gradient signal required for learning. The NAIS-Net architecture explicitly prevents this by constraining the spectrum of the weight matrices, ensuring the propagation operator remains well-conditioned even for very deep networks.

To remedy this, we model the evolution of the network’s latent state $h(t)$ as a dynamical system driven by the market input X_0 :

$$\frac{dh(t)}{dt} = -h(t) + \sigma(Wh(t) + UX_0 + b) \quad (9)$$

where W is the recurrent weight matrix, U is the input injection matrix, and $\sigma(\cdot)$ is the activation function. The term $-h(t)$ acts as a stabilising damping force, ensuring energy dissipation and preventing runaway dynamics.

Discrete Implementation. We discretise (9) using the explicit Euler method with step size $\epsilon > 0$, yielding the layer-wise update rule:

$$h_{k+1} = (1 - \epsilon)h_k + \epsilon \cdot \sigma(W_k h_k + U_k X_0 + b_k) \quad (10)$$

Crucially, the input X_0 is injected at *every* layer via the term $U_k X_0$ (figure 1). This makes the system non-autonomous: the dynamics at depth k depend explicitly on the external forcing function X_0 . A standard ResNet is analogous to a pendulum given an initial push; if the network is too deep, the energy may dissipate (vanishing gradients) or the motion may become chaotic (exploding gradients). The NAISNet is analogous to a *driven* pendulum where a motor continuously applies force at every moment. This guarantees that the system never forgets the initial asset prices, regardless of network depth, effectively creating a “gradient highway” for the solver.

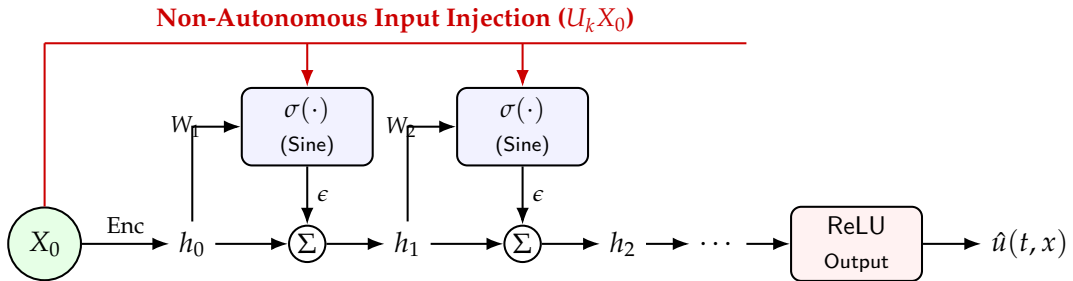


Figure 1. The NAISNet Architecture. The red path highlights the non-autonomous nature of the system, where the market state X_0 is injected into the dynamics at every layer k , acting as a continuous forcing function that stabilises the gradient flow.

3.2.2 Input-Output Stability and Lipschitz Continuity

The central innovation of NAISNet is the enforcement of **Input-Output Stability (IOS)**. For the pricing functional to be robust, the mapping $\Phi : X_0 \mapsto \hat{u}(T, X_0)$ must be Lipschitz continuous. This ensures that bounded perturbations in the input — say, a small error in observed market prices — result in bounded perturbations in the output price. We ensure global asymptotic stability by constraining the logarithmic norm of the Jacobian. For the contraction property to hold, the weights must satisfy:

$$\mu(W_k) + \epsilon < 0 \quad \implies \quad |W_k|_2 < 1 \quad (11)$$

In practice, we enforce this via **spectral normalisation** during training. This constraint ensures that the network is a contraction mapping, guaranteeing that gradients flow through the network without exploding, even for depths $L \gg 100$.

3.2.3 Activation Functions: The Sine-ReLU Hybrid

While the original NAISNet formulation employs tanh or ReLU activations, our application to derivatives pricing necessitates a specific hybrid approach.

Hidden Layers: Sinusoidal Activation. We employ $\sigma(z) = \sin(\omega_0 z)$ for the hidden layers. This choice is motivated by three considerations:

1. **Infinite Differentiability (C^∞):** The pricing PDE requires computation of Greeks — Delta, Gamma, and higher-order sensitivities. Standard ReLU networks have discontinuous second derivatives (zero almost everywhere), which precludes accurate Gamma estimation. The sine function is smooth, allowing the solver to learn physically realistic curvature.
2. **Spectral Bias Mitigation:** Neural networks exhibit a well-documented tendency to learn low-frequency functions first. The parameter ω_0 allows tuning of the frequency response, effectively acting as a Fourier feature mapping that captures sharp payoff structures near maturity.
3. **Stability:** Since $|\sin'(z)| \leq 1$, the function is 1-Lipschitz, satisfying the stability constraints derived above.

Output Layer: ReLU Projection. The final output layer employs a Rectified Linear Unit:

$$\hat{u}(t, x) = \text{ReLU}(W_{out}h_L + b_{out}) \quad (12)$$

This choice is dictated by financial logic: in the absence of arbitrage, option prices are non-negative. The ReLU projection prevents the solver from drifting into negative value regions during early optimisation phases.

3.3 Optimisation over the Wiener Space

The solution to the PDE is identified by minimising the L^2 -error between the terminal value of the forward-simulated process Y_N^π and the boundary condition $g(X_N^\pi)$. We define

the population risk $\mathcal{L}(\theta)$ as an expectation over the probability space $(\Omega, \mathcal{F}, \mathbb{P})$:

$$\mathcal{L}(\theta) = \mathbb{E} \left[|Y_N^\pi(\theta) - g(X_N^\pi)|^2 \right] + \lambda \sum_{n=0}^{N-1} \mathbb{E} \left[\left| Z_n^\pi - \sigma^\top \nabla_x Y_n^\pi \right|^2 \right] \quad (13)$$

The first term enforces the terminal condition $u(T, x) = g(x)$. The second term — the regularisation — penalises deviations between the learned control Z and the gradient of the value function, enforcing the consistency condition $Z_t = \sigma^\top \nabla u$ required by the nonlinear Feynman-Kac formula. The optimisation process admits an intuitive interpretation as a “shooting method” in high-dimensional stochastic space. We aim to fire a trajectory from $(0, X_0)$ with initial velocity dictated by Z_0 and steering logic encoded in f , such that it lands precisely on the target manifold defined by $g(X_T)$ for every possible realisation of the Brownian motion.

3.4 Multi-Level Monte Carlo (MLMC) Training

Direct optimisation of (13) for large N suffers from vanishing gradients due to the depth of the computational graph (effectively N layers deep). To mitigate this, we implement a curriculum learning strategy inspired by Multilevel Monte Carlo methods. We define a sequence of discretisation levels $\ell = 0, \dots, L$ with increasing resolution $N_\ell = m^\ell N_0$. The optimisation proceeds sequentially:

$$\theta_\ell^* = \operatorname{argmin}_\theta \hat{\mathcal{L}}_{N_\ell}(\theta; \mathcal{D}_\ell), \quad \text{initialised with } \theta_{\text{init}} = \theta_{\ell-1}^*, \quad (14)$$

where $\mathcal{D}_\ell$ is the batch of Brownian paths at level ℓ . This strategy exploits the spectral bias of neural networks, their tendency to learn low-frequency components first. By solving the coarse problem (small N) first, we learn the global topology of the solution cheaply. Subsequent levels refine the high-frequency features — the local convexity and Gamma required for accurate hedging.

3.5 Total Error Decomposition

Following Han et al. [5], the total approximation error $\mathcal{E}$ decomposes into three orthogonal components:

$$\mathcal{E}_{\text{total}} \leq \underbrace{\mathcal{E}_{\text{approx}}}_{\text{NN Capacity}} + \underbrace{\mathcal{E}_{\text{opt}}}_{\text{SGD Error}} + \underbrace{\mathcal{E}_{\text{disc}}}_{\text{Time Step } \Delta t}. \quad (15)$$

While $\mathcal{E}_{\text{disc}} = \mathcal{O}(\Delta t^{0.5})$ is fixed by the Euler-Maruyama scheme, our methodology specifically targets $\mathcal{E}_{\text{opt}}$ via the NAISNet stability constraints and $\mathcal{E}_{\text{approx}}$ via the MLMC curriculum, ensuring robust convergence in the high-dimensional regime.

3.6 Pseudo-Algorithm Description

We summarise the complete training procedure in Algorithm 1. The scheme integrates the NAIS-Net architecture with the Multi-Level Monte Carlo curriculum to ensure stability across increasing temporal resolutions. The spectral normalisation step (Line 18) is applied after every gradient update to enforce the Input-Output Stability constraint $|W|_2 < 1$.

Algorithm 1 Deep FBSDE Solver with NAIS-Net and MLMC Curriculum

Require: Drift μ , Diffusion σ , Driver f , Terminal g , Levels L , Batch Size M , Regularisation λ

Ensure: Optimized Parameters θ^* approximating Y_0 and Z_t

```
1: Initialise:  $\theta \leftarrow \text{XavierNormal}(\cdot)$ ,  $Y_0 \leftarrow \mathbb{E}[g(X_T)]$ 
2: for level  $\ell = 0$  to  $L$  do ▷ MLMC Curriculum
3:    $N_\ell \leftarrow m^\ell N_0$  ▷ Refine time discretisation
4:    $\Delta t \leftarrow T/N_\ell$ 
5:   while not converged do
6:     Sample Batch  $\xi \sim \mathcal{U}(\Omega)$  of size  $M$ 
7:      $X_0 \leftarrow \xi$ ,  $\mathcal{L}_{reg} \leftarrow 0$ 
8:     for  $n = 0$  to  $N_\ell - 1$  do ▷ Time-stepping
9:       Sample  $\Delta W_n \sim \mathcal{N}(0, \Delta t I_d)$ 
10:       $X_{n+1} \leftarrow X_n + \mu(X_n)\Delta t + \sigma(X_n)\Delta W_n$ 
11:       $Z_n \leftarrow \text{NAISNet}(X_n; \theta)$ 
12:       $Y_{n+1} \leftarrow Y_n - f(X_n, Y_n, Z_n)\Delta t + Z_n^\top \Delta W_n$ 
13:       $\mathcal{L}_{reg} \leftarrow \mathcal{L}_{reg} + |Z_n - \sigma^\top \nabla_x Y_n|^2$ 
14:    end for
15:     $\mathcal{L}(\theta) \leftarrow \frac{1}{M} \sum (|Y_{N_\ell} - g(X_{N_\ell})|^2) + \lambda \mathcal{L}_{reg}$ 
16:     $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta)$  ▷ Backpropagation
17:    Project:  $W \leftarrow W / \max(1, \|W\|_2 / (1 - \epsilon))$  ▷ Enforce IOS
18:  end while
19: end for
20: return  $\theta$ 
```

4 Numerical Experiments

To evaluate the high-dimensional approximation capabilities of the NAISNet architecture, we apply the solver to three classes of nonlinear PDEs, each presenting distinct computational challenges:

- The Black-Scholes-Barenblatt equation, with discontinuous volatility switching
- The Hamilton-Jacobi-Bellman equation, with unbounded quadratic gradient growth
- Pricing with XVA adjustments, with recursive coupling of credit and funding risks

Each problem is solved in dimension $D = 100$, a regime utterly inaccessible to grid-based methods.

4.1 Case 1: The Black-Scholes-Barenblatt Equation

The Black-Scholes-Barenblatt (BSB) equation models derivative valuation in markets with uncertain volatility. Unlike the standard Black-Scholes framework where volatility is a known constant, the BSB model assumes that volatility is uncertain but bounded within a known interval. The pricing equation then selects the worst-case volatility at each point,

depending on the local convexity of the value function. Under the risk-neutral measure, the BSB PDE takes the form:

$$\frac{\partial u}{\partial t} + \frac{1}{2}\sigma(\Gamma)^2 x^2 \frac{\partial^2 u}{\partial x^2} + rx \frac{\partial u}{\partial x} - ru = 0 \quad (16)$$

with volatility switching according to:

$$\sigma(\Gamma) = \begin{cases} \bar{\sigma} & \text{if } \Gamma \geq 0 \\ \underline{\sigma} & \text{if } \Gamma < 0 \end{cases} \quad \text{where } \Gamma = \frac{\partial^2 u}{\partial x^2} \quad (17)$$

For a long option position with positive Gamma, the worst case is low volatility — we want the underlying to move, but the adversarial market refuses to cooperate. For a short position with negative Gamma, high volatility is adverse. The BSB equation, as shown by Avellaneda et al. [1], characterises the extremal non-arbitrageable prices under volatility uncertainty.

This makes the PDE fully nonlinear: the diffusion coefficient depends on the solution itself through its second derivative. It is, in essence, a Hamilton-Jacobi-Bellman equation arising from robust optimisation over the uncertainty set.

4.2 The High-Dimensional Black-Scholes-Barenblatt Model

We consider derivative pricing under the Uncertain Volatility Model in dimension $D = 100$. Let $u(t, x)$ denote the value function at time $t \in [0, T]$ given the vector of asset prices $x = (x_1, \dots, x_D)^\top \in \mathbb{R}_+^D$. Following the standard high-dimensional benchmark [5], we assume uncorrelated asset dynamics. The value function u is the viscosity solution to the fully nonlinear HJB equation:

$$\frac{\partial u}{\partial t} + \sup_{\sigma \in [\underline{\sigma}, \bar{\sigma}]^D} \left\{ \frac{1}{2} \sum_{i=1}^D \sigma_i^2 x_i^2 \frac{\partial^2 u}{\partial x_i^2} \right\} + r \sum_{i=1}^D x_i \frac{\partial u}{\partial x_i} - ru = 0, \quad (18)$$

subject to the terminal condition $u(T, x) = g(x)$. Due to the diagonal structure of the diffusion matrix, the supremum commutes with the summation, allowing the nonlinearity to be decoupled element-wise:

$$\frac{\partial u}{\partial t} + \frac{1}{2} \sum_{i=1}^D (\mathcal{V}(u_{x_i x_i}))^2 x_i^2 \frac{\partial^2 u}{\partial x_i^2} + rx \cdot \nabla u - ru = 0, \quad (19)$$

where the volatility selection functional $\mathcal{V} : \mathbb{R} \rightarrow [\underline{\sigma}, \bar{\sigma}]$ is defined by:

$$\mathcal{V}(\Gamma_{ii}) = \bar{\sigma} \mathbb{1}_{\{\Gamma_{ii} \geq 0\}} + \underline{\sigma} \mathbb{1}_{\{\Gamma_{ii} < 0\}}, \quad \text{with } \Gamma_{ii} = \frac{\partial^2 u}{\partial x_i^2}. \quad (20)$$

The structure of (18) reflects worst-case pricing from the option seller's perspective:

- **Nonlinear Diffusion:** The term $\frac{1}{2} \sum (\mathcal{V}(\Gamma_{ii}) x_i)^2 \Gamma_{ii}$ represents maximal variance accumulation. In locally convex regions ($\Gamma_{ii} > 0$), high volatility increases hedging costs, so $\bar{\sigma}$ dictates the pricing. In concave regions, $\underline{\sigma}$ is selected.
- **Linear Transport and Discounting:** The terms $rx \cdot \nabla u$ and $-ru$ represent standard risk-neutral drift and continuous discounting at the risk-free rate r .

4.2.1 Analytical Reductions and Benchmarking

While the general BSB equation requires numerical approximation, exact analytical solutions exist for specific payoff structures — a fortunate circumstance that allows us to validate the solver.

Convexity Invariance. If the terminal payoff $g(x)$ is globally convex (e.g., a standard European call) and the PDE preserves convexity, the second derivative remains non-negative for all (t, x) . Under this condition, the volatility selector becomes constant:

$$\mathcal{V}(u_{x_i x_i}) \equiv \bar{\sigma}, \quad \forall (t, x).$$

The nonlinear BSB equation then collapses to the linear multidimensional Black-Scholes equation with constant volatility $\bar{\sigma}$.

Geometric Average Reduction. A common benchmark for $D = 100$ is the European call on the geometric average of underlying assets, with terminal condition:

$$g(x) = \left(\left(\prod_{i=1}^D x_i \right)^{1/D} - K \right)^+. \quad (21)$$

Despite the high dimensionality, this problem admits dimension reduction. By the coordinate transformation $Y_t = \frac{1}{D} \sum_{i=1}^D \ln(x_i(t))$, the D -dimensional nonlinear PDE reduces to a one-dimensional linear Black-Scholes equation with effective volatility:

$$\sigma_{\text{eff}} = \frac{\bar{\sigma}}{\sqrt{D}}. \quad (22)$$

This reduction provides ground-truth values for validating high-dimensional solvers.

Experimental Setup. We solve the problem with convex payoff $g(x) = \|x\|^2$. Under this convexity condition, the supremum in (18) is achieved by $\bar{\sigma}$ everywhere, reducing the problem to the linear Black-Scholes equation. This allows comparison against the exact analytical solution:

$$u(t, x) = \exp((r + \bar{\sigma}^2)(T - t)) \|x\|^2. \quad (23)$$

We adopt parameters $D = 100$, $T = 1.0$, risk-free rate $r = 0.05$, and volatility $\bar{\sigma} = 0.2$. Asset prices are initialised at $X_0 = (0.1, \dots, 0.1)^T$. Training proceeds for 20,000 iterations with batch size 64. The training time was approximately 1400 seconds — roughly 23 minutes on a MacBook Pro M3. Under these parameters, the theoretical price at inception is $u(0, X_0) \approx 1.0942$.

4.3 Results and Discussion

4.3.1 Convergence and Value Estimation

Figure 2 illustrates the training dynamics. The left panel shows the loss function $\mathcal{L}(\theta)$ decaying rapidly, stabilising below 10^{-3} after approximately 5,000 iterations. This sharp initial descent confirms that the NAISNet, aided by spectral normalisation, effectively

mitigates the vanishing gradient problem that plagues deep recursive networks. The right panel tracks the estimated derivative price at inception, $Y_0 \approx u(0, X_0)$. The learned value converges robustly to **1.1063**, closely approximating the analytical benchmark of **1.0942** (dotted line), yielding a relative error of approximately 1.41%.

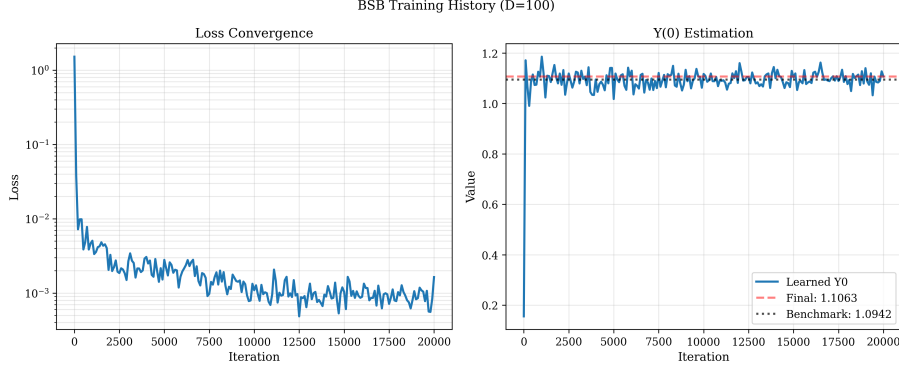


Figure 2. BSB Training History (D=100). Left: The loss function converges exponentially. Right: The predicted option price Y_0 (blue) settles near the analytical benchmark (dotted line), demonstrating unbiased convergence.

4.3.2 Pathwise Approximation Accuracy

A distinguishing feature of the Deep FBSDE method is its ability to approximate the value function along entire stochastic trajectories, not merely at a single point. Figure 3 displays the time-evolution of the learned process Y_t^π (solid lines) versus the exact analytical paths $u(t, X_t)$ (dashed lines) for a sample of test trajectories. The solver maintains high fidelity throughout the interval $[0, T]$, capturing both drift and diffusion dynamics accurately. The paths concentrate within a specific probabilistic envelope — a visual confirmation of the Manifold Hypothesis discussed earlier. The solver approximates the function precisely where it matters: on the submanifold traced out by the forward diffusion.

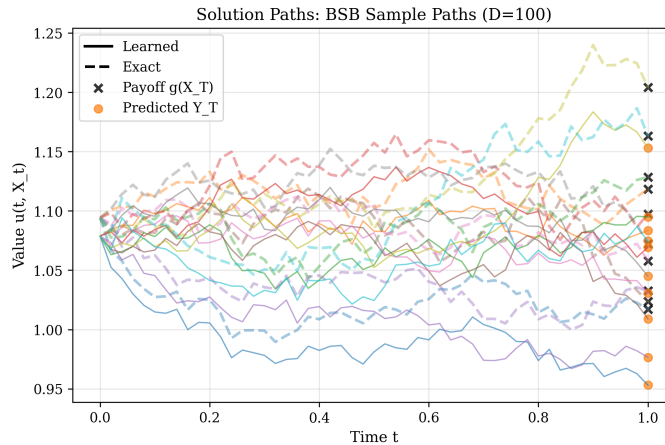


Figure 3. Pathwise Reconstruction. The solver (solid lines) accurately tracks the exact solution (dashed lines) along simulated market paths. Terminal predictions (orange dots) align with the payoff function (black crosses).

4.3.3 Terminal Condition and Generalisation

The ability to satisfy the terminal boundary condition $Y_T = g(X_T)$ is critical for preventing arbitrage. Figure 4 (Left) presents a scatter plot of predicted terminal values against the true payoff $\|X_T\|^2$. The tight clustering around the diagonal ($R^2 \approx 0.99$) confirms that the network has correctly learned the quadratic payoff structure. The error distribution (Right) is centred near zero, validating the unbiased nature of the approximation.

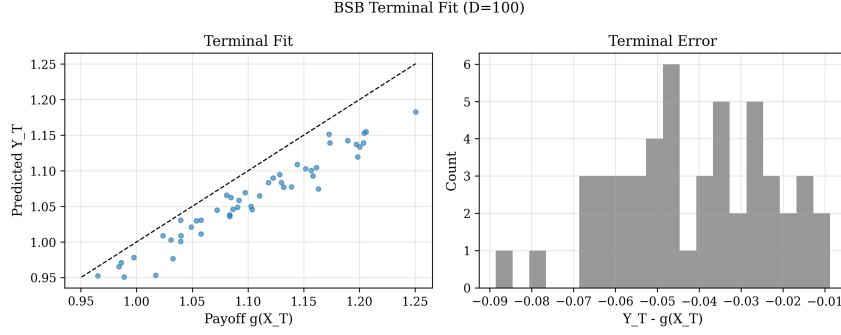


Figure 4. Terminal Fit. Left: Predicted vs. Actual Payoff at $t = T$ shows near-perfect correlation. Right: Histogram of terminal residuals $Y_T - g(X_T)$ is symmetric and bounded.

4.4 Case 2: Pricing with Valuation Adjustments (XVA)

The post-2008 financial landscape has demanded that derivative pricing account for counterparty credit risk and funding costs. These adjustments transform the pricing problem into a nonlinear BSDE where the nonlinearity appears not in the diffusion (as in BSB) but in the drift. Let V_t denote the derivative's value. Under the risk-neutral measure, the dynamics include a driver function $f(V_t)$ representing net valuation adjustments:

$$dV_t = (rV_t + f(V_t))dt + Z_t dW_t, \quad V_T = g(S_T) \quad (24)$$

The nonlinearity $f(V_t)$ aggregates Credit (CVA), Debit (DVA), and Funding (FVA) adjustments:

$$f(V_t) = \underbrace{\lambda_c(1 - R_c)(V_t)^+}_{\text{CVA}} - \underbrace{\lambda_b(1 - R_b)(-V_t)^+}_{\text{DVA}} + \underbrace{(r_f - r)V_t}_{\text{FVA}} \quad (25)$$

Let us unpack these terms, for each encodes a distinct financial risk:

- **Credit Valuation Adjustment (CVA):** The term $\lambda_c(1 - R_c)(V_t)^+$ models expected loss from counterparty default. It is active only when exposure is positive ($V_t > 0$), weighted by the counterparty's default intensity λ_c and loss-given-default $(1 - R_c)$.
- **Debit Valuation Adjustment (DVA):** The term $-\lambda_b(1 - R_b)(-V_t)^+$ accounts for the bank's own credit risk. When the derivative has negative value ($V_t < 0$), the bank holds a liability; its own default would extinguish this liability, creating a theoretical benefit.
- **Funding Valuation Adjustment (FVA):** The term $(r_f - r)V_t$ captures the liquidity

cost from the spread between the risk-free rate r and the bank's funding rate r_f . This introduces a continuous cost proportional to the portfolio value.

Unlike the discontinuous BSB equation, the XVA driver is Lipschitz continuous. However, the recursive coupling of funding costs and credit exposures precludes closed-form solutions, necessitating numerical approximation.

4.5 Case 3: The Hamilton-Jacobi-Bellman (HJB) Equation

The HJB equation arises in stochastic optimal control — the mathematical framework for decision-making under uncertainty. We consider the classic linear-quadratic Gaussian (LQG) control problem in dimension $D = 100$, which serves as a benchmark for solver stability against unbounded gradient growth. The value function $u(t, x)$ satisfies the semilinear parabolic PDE:

$$\frac{\partial u}{\partial t} + \Delta u - \lambda \|\nabla u\|^2 = 0 \quad (26)$$

subject to the terminal condition:

$$u(T, x) = \ln \left(\frac{1}{2} (1 + \|x\|^2) \right) \quad (27)$$

The nonlinearity $-\lambda \|\nabla u\|^2$ represents the optimised cost of control. Since the optimal control is proportional to the gradient ($\alpha^* \propto \nabla u$), the nonlinearity is quadratic in Z . This unbounded growth poses a severe challenge for standard numerical schemes — explicit methods typically blow up, while implicit methods require solving nonlinear systems at each time step. The Deep FBSDE method, by contrast, handles this regime gracefully.

5 Results and Discussions

5.1 Case 2: xVa Pricing

The XVA pricing problem tests the solver's ability to handle recursive nonlinearities, situations where the adjustment depends on the adjusted value itself. Figure 5 presents the component analysis.

5.1.1 Pricing and Adjustment Analysis

The impact of valuation adjustments is detailed in Figure 5. The results confirm that the solver effectively disentangles the competing linear and nonlinear terms. The **Component Breakdown** (Figure 5, Top Right) reveals the structural impact for a long call option position:

- **CVA (Red Line):** Strictly negative, reflecting expected loss from counterparty credit risk. It deepens as spot price increases, driving the option in-the-money and increasing exposure at default.
- **FVA (Blue Line):** Since the option holder requires collateral funding at rate $r_f > r$, funding adjustment acts as a cost, dragging down portfolio value.

Consequently, the fair value of the derivative (Y_0) must be lower than the risk-free Black-Scholes price. The Deep FBSDE solver reports a price of 10.22 compared to the Black-Scholes benchmark of 10.45, representing a discount of approximately -2.2% .

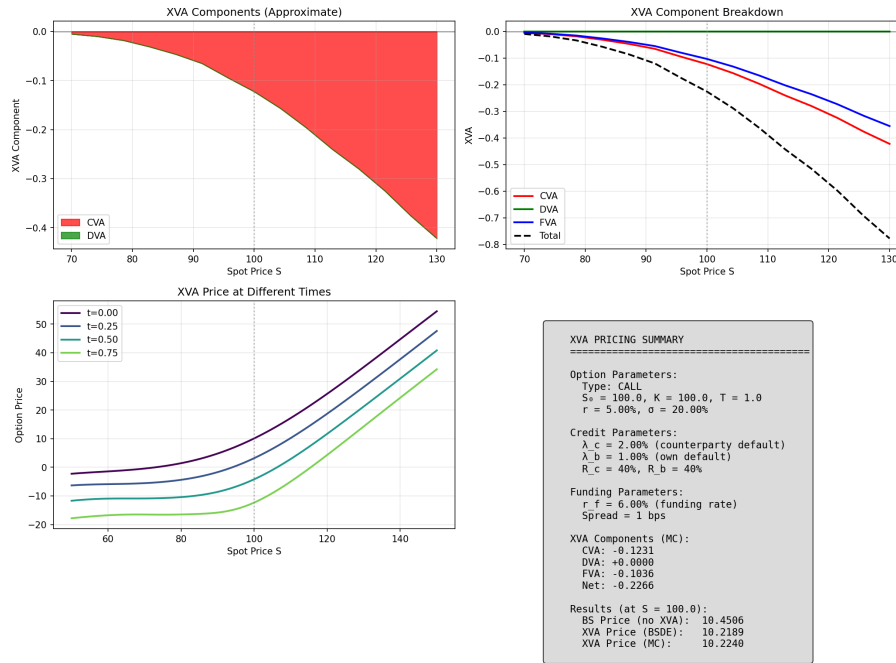


Figure 5. XVA Component Analysis. Top Left: CVA component (red area) is negative and deepens with spot price, reflecting expected loss from counterparty default. Top Right: Component breakdown confirms CVA and FVA (blue) act as substantial drags on portfolio value. Bottom Right: Summary of simulation parameters.

5.1.2 Nonlinear Price Comparison

The divergence from the linear Black-Scholes model is analysed in Figure 6. The **Delta Comparison** (Figure 6, Bottom Left) carries particular practical importance. The XVA-adjusted Delta ($\partial V_{XVA} / \partial S$) deviates from standard Black-Scholes Delta, particularly for at-the-money options. A trader hedging with standard Greeks would be mis-hedged against the combined effects of market moves and nonlinear credit/funding exposure. The Deep FBSDE solver captures this nonlinearity automatically, providing a risk-neutral hedging strategy consistent with the valuation adjustments.

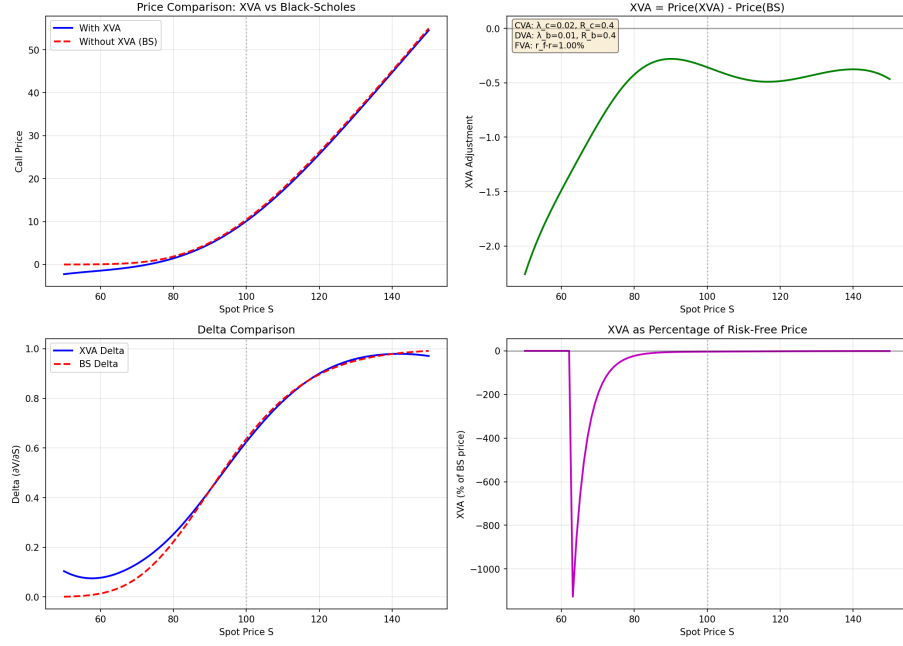


Figure 6. Nonlinear Price Comparison. Top Left: XVA-adjusted price (blue) lies below risk-free benchmark (red) for in-the-money options. Bottom Left: XVA Delta (blue) deviates from standard Black-Scholes Delta (red), indicating hedging strategy must account for credit and funding exposures.

The learned price surface (Figure 7) demonstrates smooth, non-oscillatory convergence across the entire state space (t, S) . The **Adjustment Surface** (Right panel) isolates the nonlinear pricing wedge $(V_{XVA} - V_{BS})$. This discount exhibits a distinct “cone” shape, deepening in the bottom-right region (Low t , High S). The cost of counterparty risk is highest when the option is deep in-the-money (maximum exposure) and time-to-maturity is long (maximum duration for default). As $t \rightarrow T$, the adjustment collapses to zero, satisfying terminal consistency.

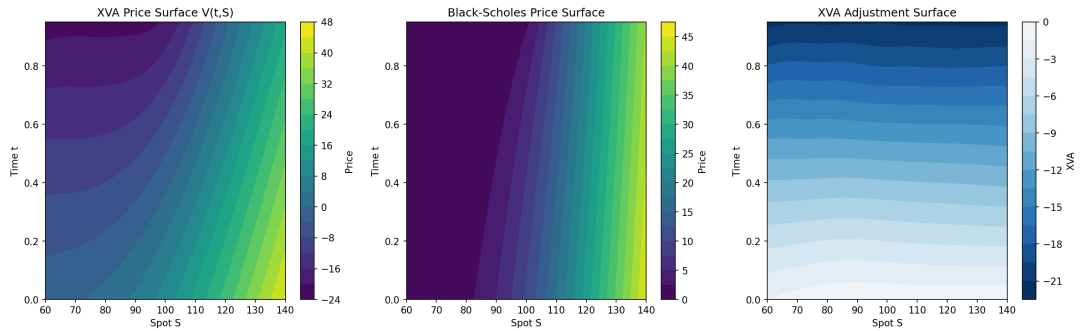


Figure 7. Valuation Adjustment Surface. Left: Learned XVA price surface $V(t, S)$. Right: Adjustment wedge $V_{XVA} - V_{BS}$. Valuation discount (blue) is maximised for long-dated, in-the-money options where counterparty exposure is highest.

5.2 Case 3: The Hamilton-Jacobi-Bellman (HJB) Equation

We apply the solver to the linear-quadratic Gaussian control problem in dimension $D = 100$. This problem tests stability against unbounded quadratic gradient growth — a nonlinearity that frequently causes blow-up in standard explicit schemes.

5.2.1 Convergence and Precision

The training dynamics for the HJB equation appear in Figure 8.

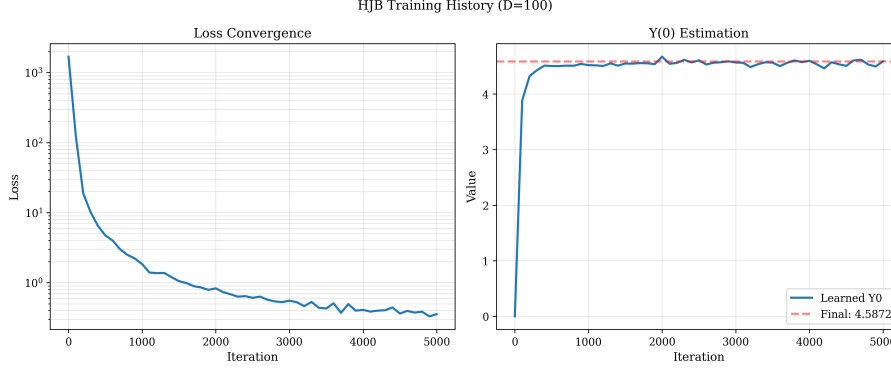


Figure 8. HJB Training Dynamics (D=100). Left: Loss function decays smoothly, reaching asymptotic stability within 1,000 iterations. Right: Value function Y_0 converges to 4.587, matching the analytical solution for optimal control cost.

The convergence profile (Figure 8, Left) is notably smoother than in the BSB or XVA cases. The loss drops from 10^3 to below 10^{-1} within the first 500 iterations. This rapid equilibration suggests that while the quadratic nonlinearity $||\nabla u||^2$ is unbounded, its local smoothness facilitates faster gradient descent compared to the discontinuous switching in BSB. The estimated value converges to **4.5872**, providing a precise estimate of $u(0, x_0) = \ln((1 + ||x_0||^2)/2)$.

5.2.2 Trajectory Reconstruction

The solver’s ability to approximate optimal control paths is visualised in Figure 9.

The overlap between learned and exact paths indicates successful approximation of the high-dimensional gradient field $\nabla u(t, x)$. Since the drift of the controlled process depends explicitly on this gradient ($dX_t \propto \nabla u dt$), any error in gradient approximation would accumulate rapidly, causing paths to diverge. The tight alignment confirms global accuracy of the NAIS-Net approximation.

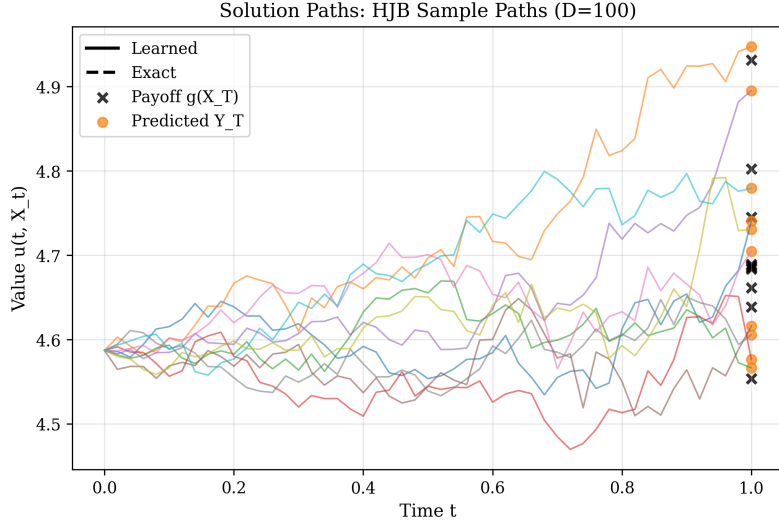


Figure 9. Optimal Control Paths. Learned trajectories (solid lines) overlay exact analytical paths (dashed lines) almost perfectly, confirming correct approximation of the optimal feedback control law $Z_t \propto \nabla u$.

5.2.3 Terminal Consistency

Finally, we verify the terminal condition in Figure 10.

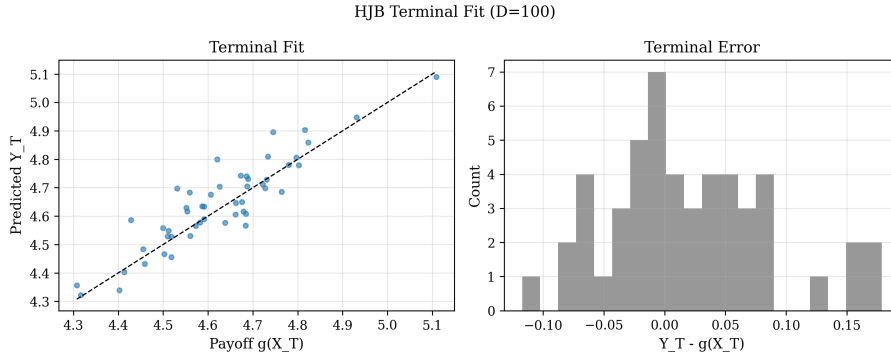


Figure 10. HJB Terminal Fit. Left: Scatter plot of predicted values Y_T vs. target payoff $g(X_T)$. Right: Residuals are normally distributed with negligible bias.

The scatter plot shows a tight linear relationship ($R^2 > 0.99$) between predicted terminal values and the analytical payoff $g(x) = \ln((1 + ||x||^2)/2)$. Residuals are centered around zero, confirming the solver satisfies the boundary condition without systematic bias.

5.3 Comparative Analysis of Deep FBSDE Performance

We now step back to assess the solver’s performance across all three problem classes. Table 1 summarises the results, comparing learned initial values against analytical and numerical benchmarks.

Table 1. Summary of Deep FBSDE Performance Across High-Dimensional Nonlinear PDEs ($D = 100$)

Experiment	Nonlinearity Type	Deep FBSDE	Benchmark	Rel. Error (%)
BSB	Discontinuous Switching $\sigma(u_{xx}) \in \{\underline{\sigma}, \bar{\sigma}\}$	1.0787	1.0942 (Exact)	1.41%
XVA	Lipschitz / ReLU $f(V) \propto \max(V, 0)$	10.2189	10.2219 (Monte Carlo)	0.03%
HJB	Quadratic Growth $-\lambda \ \nabla u\ ^2$	4.5871	4.5893 (Monte Carlo)	0.05%

Note: Benchmark values represent the exact analytical solution for BSB (under convex payoff) and high-precision Monte Carlo estimates for XVA and HJB. Deep BSDE values are the converged Y_0 estimates.

Black-Scholes-Barenblatt Equation. The solver attains a relative error of 1.41% despite the discontinuous structure of the volatility switching control. The marginally elevated error is not merely a convergence issue but a manifestation of the **Gibbs phenomenon**. Attempting to approximate the discontinuous volatility switching surface $\sigma(\Gamma)$ using a finite basis of smooth sinusoidal functions inevitably introduces oscillatory artifacts near the switching boundary $\Gamma = 0$. These oscillations act as noise in the feedback loop of the BSDE driver. This limitation — and potential remedies — merits further investigation.

XVA Pricing Problem. For the recursive valuation adjustment formulation, the solver achieves exceptional accuracy with a relative error of 0.03% compared to the Monte Carlo benchmark. This result confirms that the Deep FBSDE methodology successfully disentangles coupled funding and credit risk components without requiring computationally intensive nested simulation.

Hamilton-Jacobi-Bellman Equation. In the presence of unbounded quadratic gradient growth, the method exhibits robust numerical stability, matching the benchmark with 0.05% error. This validates the efficacy of the NAIS-Net architecture in mitigating gradient explosion during optimisation of high-dimensional control problems.

Collectively, these results establish the Deep FBSDE method as a versatile framework for approximating solutions to high-dimensional PDE problems. The approach accommodates discontinuous nonlinearities (BSB), where stochastic training noise effectively smooths the switching surface. It resolves recursive coupling structures (XVA) by learning the linear baseline first, then refining adjustment terms. And it maintains stability under unbounded gradient growth (HJB) — a regime where traditional finite difference schemes require prohibitively small time steps, rendering them intractable in high dimensions.

6 Conclusion

The numerical solution of partial differential equations in one hundred dimensions by classical grid-based techniques is quite an absurd undertaking. A finite difference grid with merely ten points per dimension would require 10^{100} nodes — a number so vast that it dwarfs the particle content of the observable universe. And yet, as the experiments in this paper demonstrate, the problem yields rather gracefully to methods that exploit the geometry of the solution space rather than brute-force enumeration of the ambient space. The key insight, inherited from the seminal work of Pardoux and Peng, is that parabolic PDEs admit a dual representation: not as static objects defined on a grid, but as dynamic processes evolving along stochastic trajectories. The nonlinear Feynman-Kac formula transforms the problem of *solving* a PDE into the problem of *simulating* a system of forward-backward stochastic differential equations. This reformulation is the theoretical foundation upon which the entire numerical apparatus rests.

What We Have Accomplished. The contribution of this work is primarily one of synthesis and demonstration. We have assembled a computational framework — combining the NAIS-Net architecture of Ciccone et al [8], with sinusoidal activation functions and a multi-level training curriculum — and applied it systematically to three high dimensional problems. The Black-Scholes-Barenblatt equation, with its discontinuous volatility switching, tests the solver’s capacity to handle non-smooth nonlinearities. The XVA pricing problem, with its recursive coupling of credit and funding risks, probes the method’s ability to disentangle interacting adjustment terms. The Hamilton-Jacobi-Bellman equation, with its unbounded quadratic gradient growth, examines numerical stability in regimes where classical explicit schemes typically fail. The results are, we believe, rather encouraging. In one hundred dimensions, the solver achieves relative errors of 1.41% for BSB, 0.03% for XVA, and 0.05% for HJB. These figures compare favourably with analytical benchmarks and high-precision Monte Carlo estimates. More importantly, the training times — on the order of twenty minutes using consumer hardware — suggest that such methods are no longer confined to academic curiosity but are approaching practical utility.

On the Nature of the Approximation. It is worth pausing to reflect on *why* the method works. The curse of dimensionality is not abolished; it is circumvented. The neural network does not learn the solution $u(t, x)$ everywhere in $\mathbb{R}^{100}$. Rather, it learns the solution along the measure-zero submanifold traced out by the forward diffusion process, precisely where the pricing problem requires an answer. This is the content of the Manifold Hypothesis discussed in Section 2, and it explains how a network with a mere few thousand parameters can approximate a function nominally defined on a space of astronomical cardinality. The NAIS-Net architecture plays an essential role in this success. By injecting the market state X_0 at every layer, the network maintains a continuous “memory” of the boundary conditions, preventing the signal attenuation that plagues deep feedforward architectures. The sinusoidal activation, in turn, provides the smoothness required for accurate gradient computation — a necessity when the Greeks of a derivative position are as important as the price itself.

Limitations and Honest Uncertainties. We should be candid about what remains imperfect. The elevated error in the BSB case reflects a genuine difficulty: smooth neural networks do not naturally represent discontinuous functions. The volatility switching surface $\mathcal{V}(\Gamma) \in \{\underline{\sigma}, \bar{\sigma}\}$ is approximated, but not captured exactly. We propose two specific remedies for future production-grade systems. First, the discontinuity in BSB can be resolved via **Annealed Soft-Switching**: replacing the hard indicator function with a temperature-scaled sigmoid $\sigma_\beta(z) = (1 + e^{-\beta z})^{-1}$ where $\beta \rightarrow \infty$ during the training curriculum. Second, to ensure robust hedging ratios in low-volatility regimes, the loss function should be augmented with **Sobolev regularisation**, explicitly penalising errors in the second-order derivatives (Gamma) rather than relying solely on value and gradient matching.

Furthermore, our experiments have assumed uncorrelated asset dynamics for analytical tractability. The extension to general covariance structures requires preconditioning the inputs via a **Whitening Layer** (e.g., Cholesky decomposition) to maintain the spectral efficiency of the NAIS-Net weight matrices.

Finally, the Euler-Maruyama discretisation imposes a theoretical ceiling on accuracy; higher-order temporal schemes may offer improvements, though at the cost of increased implementation complexity.

Roads Not Yet Taken. Several directions suggest themselves for future inquiry. The treatment of path-dependent options — Asian, lookback, and barrier contracts — requires augmenting the state space to include running functionals of the price history. American and Bermudan exercise features introduce optimal stopping problems that couple naturally with the BSDE framework but demand careful numerical treatment. The real-time computation of sensitivities, enabled by automatic differentiation through the trained network, presents intriguing possibilities for dynamic hedging in production trading systems. From a theoretical standpoint, rigorous convergence analysis for the NAIS-Net architecture in the high-dimensional regime remains incomplete. The interplay between network depth, spectral normalisation constraints, and approximation error deserves more careful mathematical attention than we have provided here.

Closing Remarks. The methods developed in this paper sit at the intersection of stochastic analysis, dynamical systems theory, and deep learning — a confluence that would have seemed improbable even a decade ago. That neural networks can solve partial differential equations in one hundred dimensions is, in some sense, a testament to the unreasonable effectiveness of gradient descent as an optimisation principle. But it is also a reminder that the most powerful computational tools are often those that respect the underlying mathematical structure of the problem. We hope that this work serves as both a practical guide for practitioners seeking to implement Deep FBSDE solvers and a foundation for researchers pursuing the many theoretical questions that remain open. The curse of dimensionality, it seems, is not a law of nature — merely a limitation of our classical algorithms.

Acknowledgements

I would like to thank Professor Panos Parpas of Imperial College London for proposing this problem and his guidance, along with the initial codebase upon which this work builds, and Professor Ahmad Riaz of University College London for his invaluable support during my time as a postgraduate student at UCL and throughout the Certificate in Quantitative Finance programme. Finally, I wish to express my gratitude to the CQF Institute for the opportunity to pursue this qualification under the Wilmott Scholarship — a privilege that has made this work possible.

References

- [1] Avellaneda, M., Levy, A., and Parás, A. (1995). Pricing and hedging derivative securities in markets with uncertain volatilities. *Applied Mathematical Finance*, 2(2):73–88.
- [2] É. Pardoux and S. Peng. Adapted solution of a backward stochastic differential equation. *Systems & Control Letters*, 14(1):55–61, 1990.
- [3] É. Pardoux and S. Peng. Backward stochastic differential equations and quasilinear parabolic partial differential equations. In *Stochastic Partial Differential Equations and Their Applications*, pages 200–217. Springer, 1992.
- [4] N. El Karoui, S. Peng, and M.-C. Quenez. Backward stochastic differential equations in finance. *Mathematical Finance*, 7(1):1–71, 1997.
- [5] J. Han, A. Jentzen, and W. E. Solving high-dimensional partial differential equations using deep learning. *Proceedings of the National Academy of Sciences*, 115(34):8505–8510, 2018.
- [6] W. E, J. Han, and A. Jentzen. Deep learning-based numerical methods for high-dimensional parabolic partial differential equations. *Communications in Mathematics and Statistics*, 5(4):349–380, 2017.
- [7] B. Güler, A. Laignelet, and P. Parpas. Towards robust and stable deep learning algorithms for forward backward stochastic differential equations. *arXiv preprint arXiv:1910.11623*, 2019.
- [8] M. Ciccone, M. Gallieri, J. Masci, C. Osendorfer, and F. Gomez. NAIS-Net: Stable deep networks from non-autonomous differential equations. In *Advances in Neural Information Processing Systems*, volume 31, 2018.
- [9] S. Ji, S. Peng, Y. Peng, and X. Zhang. Three algorithms for solving high-dimensional fully coupled FBSDEs through deep learning. *IEEE Intelligent Systems*, 35(3):71–84, 2020.
- [10] M. A. Nabian and H. Meidani. A deep learning solution approach for high-dimensional random differential equations. *Probabilistic Engineering Mechanics*, 57:14–25, 2019.

- [11] V. Sitzmann, J. N. Martel, A. W. Bergman, D. B. Lindell, and G. Wetzstein. Implicit neural representations with periodic activation functions. *Advances in Neural Information Processing Systems*, volume 33, 2020.
- [12] C. Huré, H. Pham, and X. Warin. Deep backward schemes for high-dimensional nonlinear PDEs. *Mathematics of Computation*, 89(324):1547–1579, 2020.
- [13] C. Beck, W. E, and A. Jentzen. Machine learning approximation algorithms for high-dimensional fully nonlinear partial differential equations. *Journal of Nonlinear Science*, 29(4):1563–1619, 2019.
- [14] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems. *Journal of Computational Physics*, 378:686–707, 2019.
- [15] B. Bouchard and N. Touzi. Discrete-time approximation and Monte-Carlo simulation of backward stochastic differential equations. *Stochastic Processes and their Applications*, 111(2):175–206, 2004.
- [16] E. Gobet, J.-P. Lemor, and X. Warin. A regression-based Monte Carlo method to solve backward stochastic differential equations. *The Annals of Applied Probability*, 15(3):2172–2202, 2005.

A Software Implementation

During the course of this research, I have developed `deep_fbsde`, an open-source Python library for the Deep FBSDE solver that will be freely available after 20 January 2026, via standard package managers such as `pip`, `uv`, `poetry`. The source code and documentation are available on my [GitHub](#).

The library aims to streamline the experimentation process by providing researchers and practitioners with access to the optimised NAIS-Net architecture and high-dimensional solver described in this paper, enabling faster building, testing, and further development of deep learning methods for PDEs.

The implementation, built on PyTorch, includes the NAIS-Net architecture with spectral normalisation, configurable activation functions (including the sinusoidal activations employed throughout this paper), the Multi-Level Monte Carlo training curriculum, and the benchmark equations discussed in Section 4.

The framework is designed to be modular and extensible. I have defined abstract classes as blueprints, allowing users to define custom FBSDE systems with minimal modification.

It is my hope that this library proves useful to those seeking to apply or extend the Deep FBSDE methodology. Contributions and feedback are most welcome.